Athena Python Endterm Cheatsheet

Introduction to python (Universiteit van Amsterdam)

AthenaStudies

INTRODUCTION TO PYTHON REFERENCE (ENDTERM VERS.) *to only be used for academic purpose

Object	**Structure**	**Function to turn other objects into it**
Integer	… or -1 or 0 or 1 or …	**int(O)**
Floating-point number	… or -1.2 or 0 or 1.2 or …	**float(O)**
String (characters)	**"…"** or **'…'**	**str(O)**
Tuple	**(O1, O2, O3, …)**	**tuple(O)**
List	**[O1, O2, O3, …]**	**list(O)**
Dictionary	**{O1: value1, O2: value2, O3: value3, …}**	**dict(O)**
Boolean	**True** or **False**	**bool(O)**

Basic Input/Output Mechanisms
input(msg) prints msg and then returns the value entered as a string.
print(s1, …, sn) prints strings s1, …, sn separated by spaces.

4 Control Flows
if cond: … elif cond: … else: … executes the actions below the first-met **True** condition.
while cond: … is the loop that allows executing a task as long as the condition is **True**.
for v in seq: … loops through the sequence with v as the element (key for dict) in each iteration.
func(arg1, …) or **O.meth(arg1, …)** takes in some inputs, do a set of actions, and returns some outputs

Common Operations on Numeric Types (int & float)
i + j is the sum of i and j.
i – j is i minus j.
i * j is the product of i and j.
i / j is floating-point division. (10/2 → 5.0)
i // j is floor division. (10//2 → 5; 11//2 → 5)
i%j is the remainder when the int i is divided by the int j.
ij** is i raised to the power j.

Common Operations on Iterable/Sequence Types (str, tuple, list, dict, …)
seq[i] returns the i^{th} element in the sequence.
len(seq) returns the length of the sequence.
seq1 + seq2 concatenates the two sequences. (Not available for range.)
n*seq returns a sequence that repeats seq n times. (Not available for range.)
seq[start:end:step] returns a sequence that is a slice of seq.
e in seq tests whether e is contained in the sequence.
e not in seq tests whether e is not contained in the sequence.
for e in seq iterates over the elements of the sequence. (Except for dict, in which e will iterates over its keys)

Boolean Operators
x == y returns True if x and y are equal.
x != y returns True if x and y are not equal.
<, >, <=, >= have their usual meanings.
a and b is True if both a and b are True, and False otherwise.
a or b is True if at least one of a or b is True, and False otherwise.
not a is True if a is False, and False if a is True.

4 Functions That Return an Iterable Object
map(function, seq) returns a sequence in which elements from the old sequence are inputted to the function.
zip(*seq) returns a sequence by combining all inputted sequences into 1 by elements.
range(start=0, end, step=1) returns a sequence of numbers from start to end, in-/decremented by step.
enumerate(seq) returns a sequence with elements of the form (i^{th} index, old sequence's i^{th} element).

List Methods
L.append(e) adds the object e to the end of L.
L.count(e) returns the number of times that e occurs in L.
L.insert(i, e) inserts the object e into L at index i.
L.extend(L1) appends the items in list L1 to the end of L.
L.remove(e) deletes the first occurrence of e.
L.index(e) returns the index of the first occurrence of e.
L.pop(i=-1) removes and returns the item at index i.
L.sort() has the side effect of sorting the elements of L.
L.reverse() reverses the order of elements in L.
L.copy() returns a shallow copy of L.
L.deepcopy() returns a deep copy of L.

String Methods
s.count(s1) counts how many times the string s1 occurs in s.
s.find(s1) returns the index of the first occurrence of the substring s1 in s; returns -1 if s1 is not in s.
s.rfind(s1) same as find, but starts from the end of s.
s.index(s1) same as find, but raises an exception if s1 is not in s.
s.upper() or **s.lower()** converts all letters to upper- or lowercase.
s.replace(old, new) replaces all occurrences of string old with string new.
s.strip() or **s.rstrip()** or **s.lstrip()** removes trailing white spaces in front of and behind s.
s.split(d) splits s using d as a delimiter and returns a list of substrings of s.

Object-Oriented Programming **Class** is a group of similar objects **Instance** is an object in a class **O.name** is an attribute of an object **O.name(…)** is a method of an object	**pandas Library** **import pandas as pd** **pd.Series(seq, index=None)** - **S[i]** is indexing - **S[condition]** selects values based on condition - **S.index** gives the current indices - **S.map(function)** applies function to every value in S **pd.DataFrame(seqOfColumns, index=None, columns=None)** - **D [i]** is column indexing - **D.loc[row, col=None]** is indexing based on labels - **D.iloc[row, col=None]** is indexing based on default indices - **D.index** gives the current row indices - **D.set_index(seq)** sets new row indices - **del D[col]** deletes column - **D.pop(col)** deletes column and returns it - **D1.append(D2)** adds D2 to the bottom of D1 - **D.sort_index(ascending=True)** creates new D based on sorted indices - **D.sort_values(col, ascending=True)** creates new D based on sorted column - **D.drop(row)** deletes row - **D.dropna(how='…')** removes rows with missing values - **D.fillna(value, method='…', limit=n)** fills missing values
datetime Library **from datetime import datetime, timedelta** **datetime.now(tz=None), datetime.today(), datetime.utcnow()** return current date and time **datetime.strptime("…", "format")** creates datetime object from string based on format **datetime(year, month, day, hour=0, minute=0, second=0, tzinfo=None)** - **d.year, d.month, d.day, d.hour, d.minute, d.second** are its attributes - **d.strftime("format")** creates a string based on format **timedelta(days=0, seconds=0, minutes=0, hours=0, weeks=0)** - **t = datetimeO1 – datetimeO2** - **t.days, t.seconds** are its attributes	
pytz Library **import pytz** **pytz.timezone("…")**	

Miscellaneous **f"…{expression}…"** is f-string **[expr for e in iterableO if condition]** is list comprehension **{key: value for e1, e2 in iterableO if condition}** is dictionary comprehension